POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Extended SayCan for Enhanced LLM-Guided Robot Planning

Technical report
Machine Learning for Mechanical Systems

Matteo FILIPPI - 10766763
Alessandro GORLA - 10727569
Luca MASTRORILLO - 10667759

Professor: Prof. Loris Roveda
Academic Year: 2024-25

Contents

Contents **i**

1 Introduction **1**

2 Environment and Object Detection Enhancement **2**
2.1 Extension of Object Types in the Environment 3
2.2 Object Recognition Validation using ViLD 5

3 LLM grounding **9**
3.1 Motivation 9
3.2 Implementation 9
3.3 Use cases examples 11
3.4 Advantages and Limitations 15

4 Feedback and Plan Adjustment **16**
4.1 Implementation 17
4.2 Advantages of Iterative Feedback 18
4.3 Example 1 19
4.4 Example 2 23

5 Conclusions **24**

1 | Introduction

SayCan is a framework that combines Large Language Models (LLMs) with robotic, enabling robots to follow natural language commands by filtering and prioritizing possible actions based on both language cues and affordance scores.

This project builds on an existing implementation of the SayCan framework, aiming to control a robot in a simulated environment using an LLM. The LLM receives as input a list of predefined actions, such as picking up and placing objects—and outputs a probability distribution that estimates the appropriateness of each action as the next step. SayCan enhances this process by incorporating affordance information, which represent the feasibility of certain commands based on the comparison between user-provided inputs and the current state of the environment.

Robot actions are guided by CLIPort framework, a model that leverages on machine learning to perform pick-and-place operations based on visual input and language commands.

The goal of this project was to modify and extend the behavior of SayCan by improving some critical aspect of its performance. The proposed extensions focused on the following areas:

- Environment and Object Detection Enhancements
- Advanced LLM Grounding
- Feedback and Plan Adjustment

Technical Components

ViLD: Zero-shot object detection from top-down RGB image.

xyz-map: Obtained via projection from the depth buffer; maps image pixels to PyBullet world coordinates.

Scene Description: A structured text listing object types, colors, and estimated (x,y) positions.

LLM: GPT model scoring action candidates given updated scene context.

2 | Environment and Object Detection Enhancement

The initial environment configuration was based on the creation and positioning of two categories of objects: bowls (fig. 1a) and blocks (fig. 1b). Each object is associated with a color, selected from five different options (blue, red, green, yellow, orange). Additionally, "block" objects can be classified either as a *pick* or a *place* object, while "bowl" object is only conceived as a container for placing other objects.

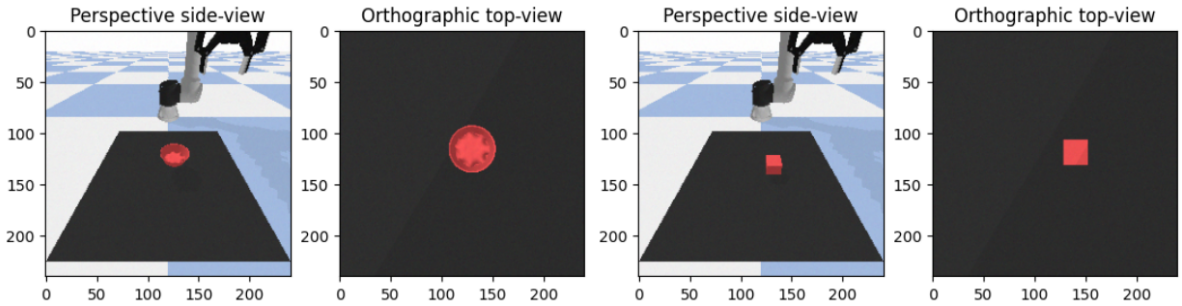


Figure 1: Pre-existing solid object in SayCan environment: a) red bowl object b) red block object

The management and simulation of these solid objects within the SayCan environment are handled by the `PyBullet` physics simulation library. `PyBullet` provides built-in functionalities for generating, positioning, and rendering basic geometric solids, enabling accurate physical interactions and realistic visualization. A detailed description of `PyBullet`'s internal workflow and operational logic is provided in the following section, highlighting how the library interprets geometric and physical inputs to simulate dynamic interactions between objects effectively.

`PyBullet`: Detailed Workflow and Operating Logic

`PyBullet` is an advanced physics-based simulation library specifically designed for the accurate modeling and analysis of robotic systems and dynamic interactions within complex environments. At its core, `PyBullet` leverages numerical integration techniques and efficient collision detection algorithms to simulate realistic physical behavior in real-time or offline simulations.

The typical workflow of a `PyBullet` simulation follows a structured and clearly defined process, which can be divided into three main phases: initialization, simulation, and output generation.

Initialization Phase

During the initialization phase, `PyBullet` requires input definitions that comprehensively describe the simulated entities. These definitions can be provided through:

- **`.urdf` files**: Unified Robot Description Format files that contain detailed geometric, inertial, visual, and collision information about robotic mechanisms and custom solids.
- **Built-in geometric primitives**: Standard shapes provided directly by `PyBullet`, such as spheres, cylinders, and boxes, which can be instantiated through simplified function

calls without the need for external geometry files.

The library subsequently parses these inputs and builds an internal representation of each object's geometry, mass distribution, inertia tensor, frictional properties, and visual characteristics. `PyBullet` also allows the explicit specification of environmental factors, including gravitational acceleration, frictional coefficients, and restitution values, thus providing users fine-grained control over the simulated conditions.

Simulation Phase

Once initialization is complete, `PyBullet` advances the simulation by numerically integrating the equations of motion. Specifically, `PyBullet` performs the following actions at each simulation step:

- Computes internal dynamic states of objects, updating positions, velocities, and accelerations based on external forces, constraints, and interactions.
- Detects collisions and calculates the appropriate collision responses using robust physics-based algorithms to ensure physically accurate outcomes.
- Resolves articulated body dynamics, handling joint constraints and degrees of freedom defined in robot descriptions (`.urdf`).

Output Generation Phase

After simulation, `PyBullet` outputs comprehensive data for analysis and visualization, including:

- Kinematic and Dynamic Data: Positions, orientations, velocities, accelerations, and applied forces for each entity at discrete simulation steps.
- Collision Information: Detailed reports of collision events, specifying involved objects, contact points, and forces exchanged.
- Visual Outputs: Rendered images or video sequences that visually illustrate the simulation process, enabling detailed qualitative assessment of the simulation fidelity.
- Data Logs: Structured data outputs that can be stored and exported for offline analysis or integration into external workflows.

Overall, `PyBullet` integrates structured inputs, robust physical computations, and detailed output logging to deliver accurate and reliable physics simulations. Its modularity and extensive feature set make it particularly suited for robotics research, system validation, reinforcement learning environments, and prototyping complex interactions within virtualized physical scenarios.

2.1. Extension of Object Types in the Environment

The proposed extension involves the addition of two new classes of solid figures to be placed in the environment. Within the developed environment, a subset of solids provided natively by the `PyBullet` library has been implemented, namely spheres, cylinders, and prisms. These built-in solids have predefined geometric and physical properties, allowing users to instantiate them directly without requiring external geometry files. Specifically, `PyBullet` provides the following standard solids:

- **Sphere**: Defined by a single parameter, its radius. `PyBullet` internally computes its geometric representation, collision boundaries, and inertia properties based on this radius.
- **Cylinder**: Characterized by two parameters, radius and height. Its geometry, collision shape, and inertia are automatically computed by `PyBullet` from these input dimensions.
- **Block (Cube)**: Defined by three parameters representing the half-extents along the x, y, and z axes. `PyBullet` generates the complete geometric shape and associated collision boundaries directly from these half-extents.

Solid Type	Dimensional Parameters	Description
Sphere	Radius (r)	Single dimension defining the sphere radius
Cylinder	Radius (r), Height (h)	Radius and height defining cylindrical shape
Block (Box)	Half-extents (x, y, z)	Dimensions along x, y, z axes defining rectangular box

Table 1: Dimensional description of built-in `PyBullet` solids

For each of these built-in objects, `PyBullet` provides consistent and optimized routines to handle physical interactions, collision detection, and rendering, thereby ensuring robust and efficient simulations. This facilitates seamless integration and rapid prototyping within simulation frameworks such as SayCan, without the need for custom geometric definition files.

To further expand the set of available objects, a dimensional differentiation was introduced for solids generated via `pybullet`. Two size categories were defined: "small" and "big", with geometry parameters specified in the "Setup Environment" section. These parameters vary depending on the type of solid they refer to (see tab. 2).

Solid Type	Size "Small"	Size "Big"
Block [length, width, height]	[0.015, 0.015, 0.015] m	[0.025, 0.025, 0.025] m
Sphere [radius]	[0.017] m	[0.035] m
Cylinder [radius, height]	[0.015, 0.015] m	[0.025, 0.030] m

Table 2: Size parameters for solid objects

Additionally, the following solids were introduced into the environment. To enable the integration of complex geometric solids into the SayCan simulation environment, each object has to be represented using three standard 3D model file formats. These are listed below:

- `.obj`: Defines the geometry of the solid, including its vertices and faces, allowing the physical shape to be rendered within the `PyBullet` environment.
- `.mtl`: Provides material and visual information such as surface colors, reflectivity, and texture mapping, which are essential for visually distinguishing between different objects.
- `.urdf` (Unified Robot Description Format): Serves as the configuration file for `PyBullet`, linking the geometric and visual information while also specifying additional properties such as mass, collision behavior, and the inertial frame of the object.

These three files together are necessary to ensure that the custom solids are correctly interpreted and rendered within the SayCan framework, enabling the simulation to perform object manipulation tasks with consistent visual and physical properties.

These files were obtained from an online open dataset. The newly introduced solids are:

- Hexagonal-base prism
- Triangular-base prism
- Star-shaped base prism

For these latter objects, only a single size has been maintained, both due to the difficulty in dynamically managing their scale and to introduce heterogeneity in the object descriptions.

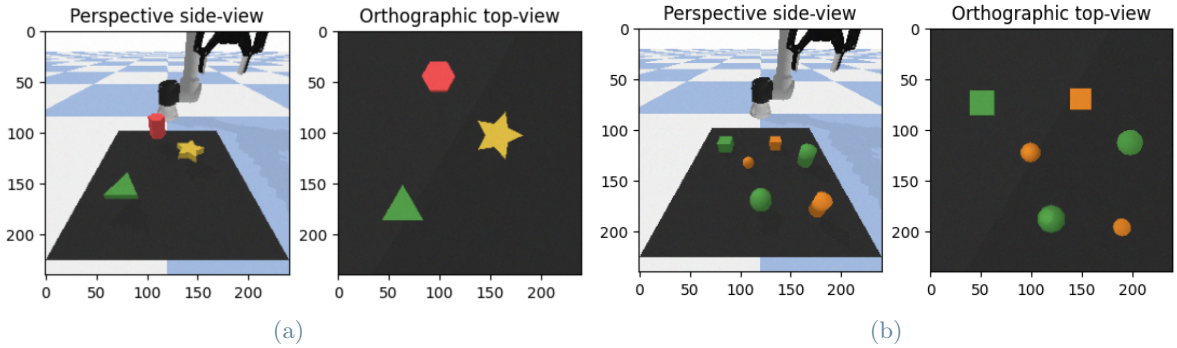
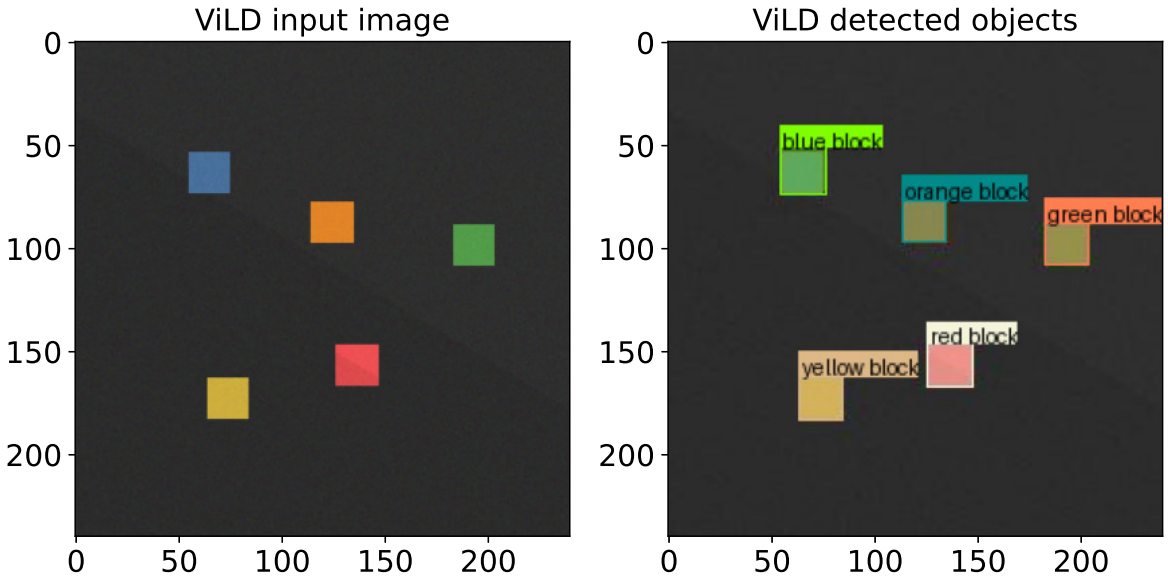


Figure 2: Newly imported solid figures in SayCan environment: a) hexagonal-base prism (red), star prism (yellow), triangular-base prism (green) b) small sized block, cylinder and sphere (orange) and big sized block, cylinder and sphere (green)

2.2. Object Recognition Validation using ViLD

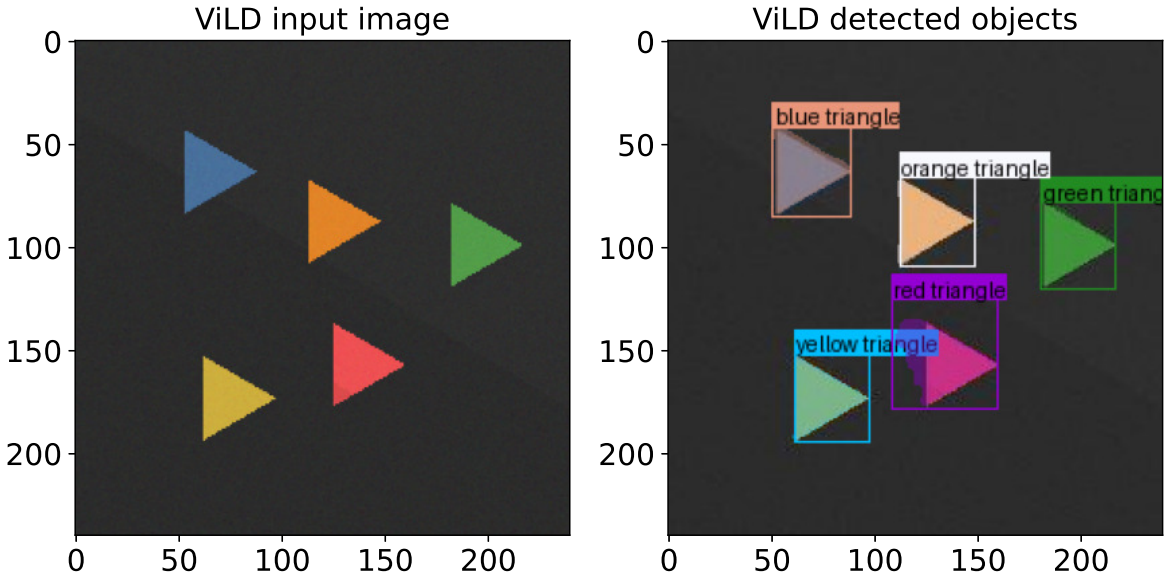
A validation was carried out to assess ViLD's ability to detect objects in various scenarios: ViLD successfully identified the presence of objects and correctly recognized and labeled them by shape and color, as shown in figs. 3 to 8. Below is the list of the tested scenarios:

- Same-shape scenario: varied combinations of shapes and colors
- Same-color scenario: different shapes sharing the same color
- Mixed scenario: different combination of shapes and color



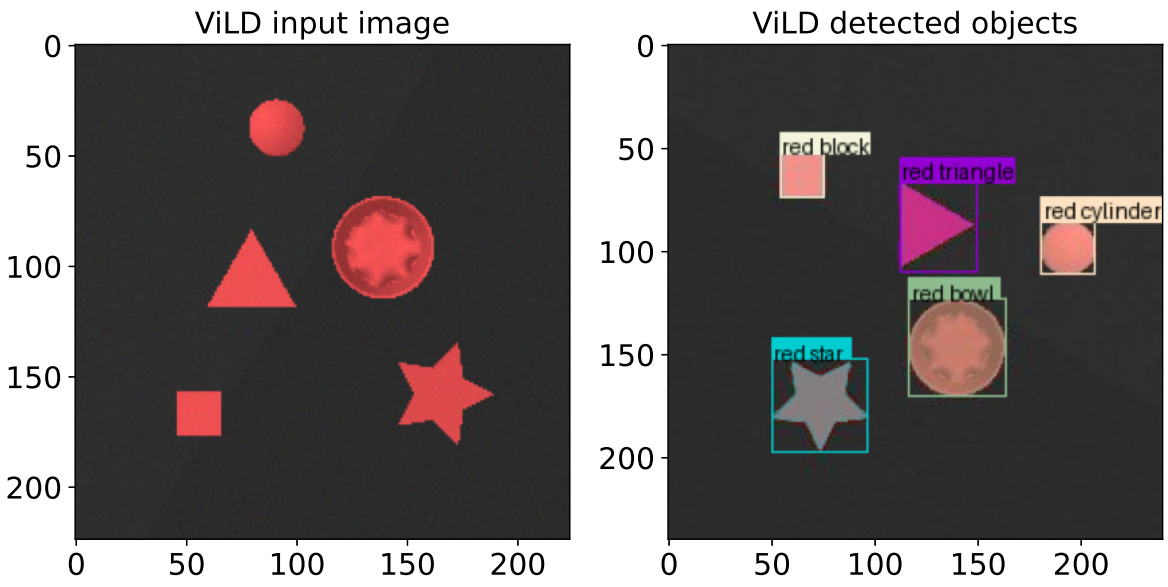
(a) (b)

Figure 3: ViLD recognition for first same-shape scenario a) input image b) output image



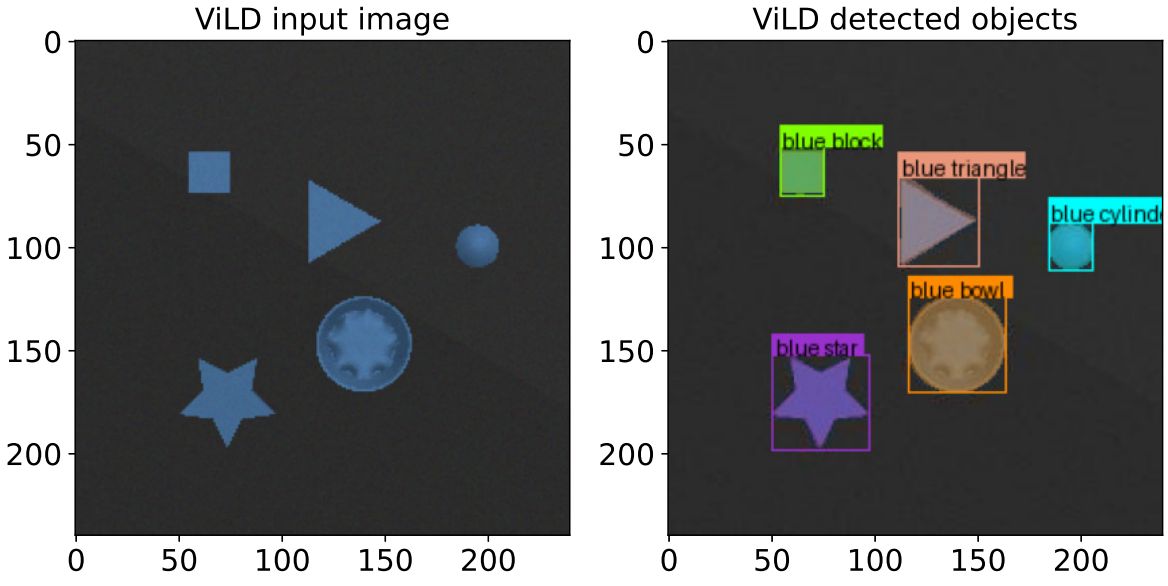
(a) (b)

Figure 4: ViLD recognition for second same-shape scenario a) input image b) output image



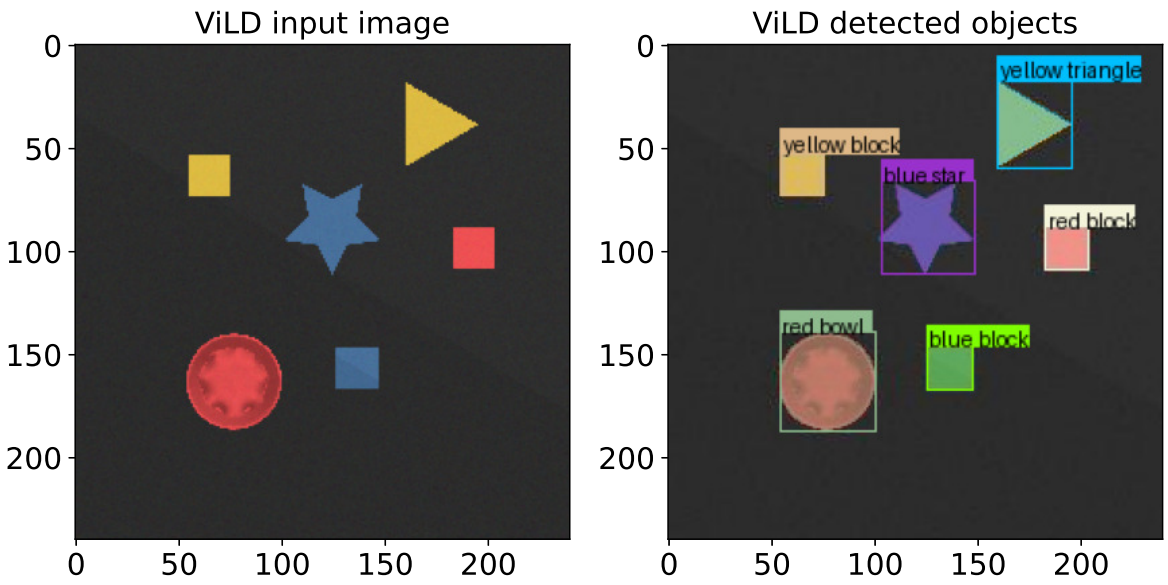
(a) (b)

Figure 5: ViLD recognition for first same-color scenario a) input image b) output image



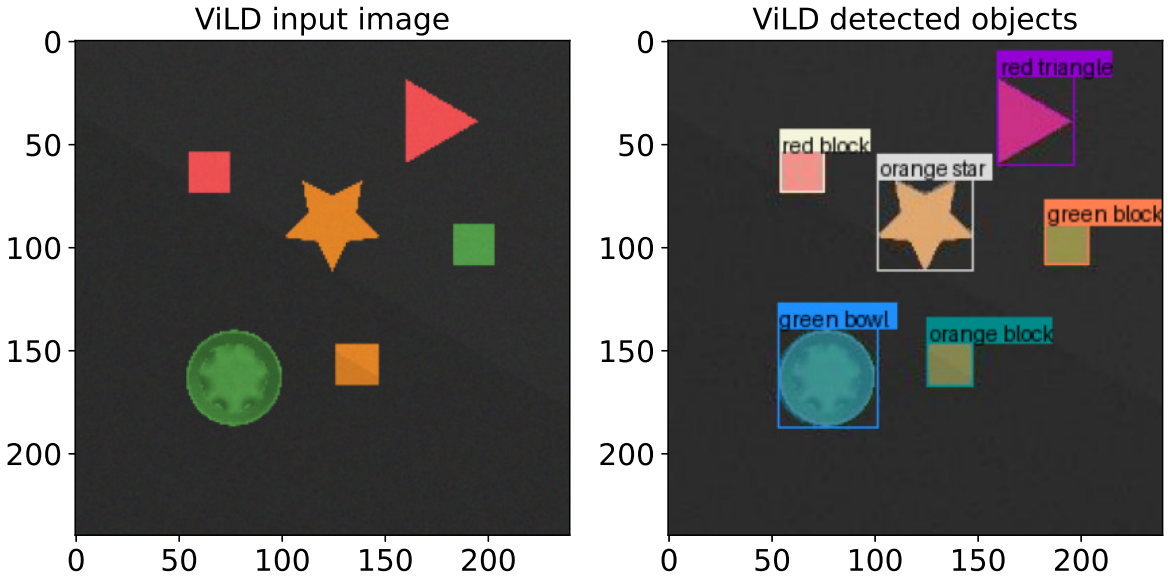
(a) (b)

Figure 6: ViLD recognition for second same-color scenario a) input image b) output image



(a) (b)

Figure 7: ViLD recognition for first mixed scenario a) input image b) output image



(a) (b)

Figure 8: ViLD recognition for second mixed scenario a) input image b) output image

3 | LLM grounding

In our implementation of the SayCan-inspired framework, we introduced a key extension to the language-vision-action pipeline: the inclusion of spatial information, namely the 2D position of objects in the scene as an additional input to the language model.
This modification enhances the model's ability to ground high-level natural language commands in a physical environment where spatial context plays a critical role in task disambiguation and prioritization.

3.1. Motivation

Originally, the SayCan framework interprets user-issued prompts (e.g., "Pick up the red block and place it on the blue one") purely based on semantic content and a predefined list of action primitives. While this is sufficient for many well-specified tasks, it presents limitations in more complex situations where relative positioning or geometric arrangement of objects influences task interpretation. For instance:

- Pick the object closest to the red block.
- Stack the blocks from left to right.
- First move the farthest object, then the nearest one.

Such instructions inherently require positional reasoning, which are not typically provided by textual labels alone. To support this capability, we equipped the language model with additional structured input representing the location of each recognized object in the scene.

3.2. Implementation

The positional information is obtained via ViLD, which is used to recognize and localize objects in the top-down image of the environment. Once ViLD classifies and segments the visible objects using a set of category prompts, the following steps are performed:

1. **Centroid Computation:**
   To estimate the position of each detected object, we compute the centroid of its associated bounding box in the 2D image plane. To this end, we modified the original ViLD processing function to produce an additional output variable. The function iterates through the detection results and, for each recognized object, extracts the bounding box coordinates provided by the model.

   Once the bounding box for a given object is retrieved, we compute its centroid by taking the average of the top-left and bottom-right corner coordinates:

   $$x_c = \frac{x_{\min} + x_{\max}}{2}, \quad y_c = \frac{y_{\min} + y_{\max}}{2}$$

   where $(x_{\min}, y_{\min})$ and $(x_{\max}, y_{\max})$ define the bounding box.

   This yields a pair of pixel coordinates (x_c, y_c) for each object, representing the estimated center of its spatial extent in the image.
   In the next steps, we will address how to convert these pixel-based centroids into coordinates in the cartesian reference frame. Thus, the newly added output consists of a

dictionary (a) that associates each detected object class with its corresponding cartesian coordinates.

2. **Coordinate Transformation:**
The pixel-based centroids computed in the previous step must be converted into real-world spatial coordinates to be actionable by the robot. This transformation is possible because the camera is mounted in a fixed, orthographic, top-down view, and it is perfectly centered with respect to the square workspace.

   In particular, we know the following quantities:

   - the resolution of the acquired image (i.e., number of pixels per side),
   - the actual physical length of the square workspace (e.g., 0.6 x 0.6 m),
   - the position and orientation of the pixel reference frame relative to the global Cartesian reference frame.

   In particular, we know that the origin of the pixel reference frame lies at coordinates $(-0.3\,m, -0.5\,m)$ in the global Cartesian frame, and that the pixel frame is rotated with respect to the global frame. Therefore, to convert the pixel coordinates into world coordinates, we first apply a rigid roto-translation to account for both the shift in origin and the orientation misalignment.

   After applying the roto-translation, we scale the resulting coordinates using a constant factor derived from the known geometry. Let N be the number of pixels per image side, and let L be the real-world length of the square workspace. Then, the scale factor is defined as:

$$s = \frac{L}{N} = \frac{0.6\,m}{240\,px} = 0.0025\,\frac{m}{px}$$

   Thus, each pixel coordinate is first roto-traslated and then multiplied by s to obtain the corresponding (x, y) position in meters. This yields an accurate mapping from pixel space to the robot's world frame.

   This coordinate transformation is essential for the correct execution of manipulation tasks, as it bridges the gap between vision-based detection (in pixel space) and physical robot actions (in metric space). In our system, this spatial information is not directly used to control the robot's motion (e.g., for inverse kinematics or grasping), but it is crucial for high-level language understanding tasks (e.g., deciding which object is closer to another, or resolving spatial conditions in the prompt).

3. **Integration into the Prompt:**
Once each object has been detected and its real-world coordinates have been computed, this spatial information is encoded into a natural language string and appended to the prompt given to the language model. Such positions are concatenated into a human-readable description in the format:

   ```
   <label> → is positioned in the environment at these coordinates (x,y):  (<x_pos>,
   <y_pos>)
   ```

   These strings are collected into a single variable called `obj_pos_string`, which represents the positional map of all detected objects.

The full environment description is constructed by combining:

(a) A high-level scene summary, produced by `build_scene_description()`,

(b) A spatial frame reference (i.e., table dimensions and corner positions),

(c) The object position string `obj_pos_string`.

The final description is wrapped between `BEGIN SCENE DESCRIPTION` and `END SCENE DESCRIPTION` markers, and appended both to the `commands_string` used in scoring and to the GPT prompt if spatial grounding is enabled.

```
BEGIN SCENE DESCRIPTION:
The objects are placed on a table of dimensions 0.6 and 0.6.
Top-left corner is at (-0.25, -0.25).
Top-right corner is at (0.25, -0.25).
Bottom-left corner is at (-0.25, -0.75).
Bottom-right corner is at (0.25, -0.75).
red block → is positioned in the environment at these coordinates (x,y):
(0.12, 0.35)
blue block → is positioned in the environment at these coordinates (x,y):
(0.07, 0.43)
END SCENE DESCRIPTION.
```

This enriched prompt is passed to the language model together with the original user query and the set of candidate actions. The model evaluates all actions by computing a relevance score via `gpt3_scoring()`, later modulated by affordance scores from a parallel scoring function. The final action is selected by combining language and affordance scores, and the selected command is used for subsequent execution with CLIPort.

This integration of spatial context allows the system to better resolve prompts that depend on object arrangement, distance, or spatial conditions.

3.3. Use cases examples

To validate the utility of integrating object positions into the language model prompt, we tested the system on a variety of spatially grounded tasks. Below we report two representative examples that demonstrate how spatial awareness enables the LLM to resolve otherwise ambiguous instructions.

Example 1: Spatial Proximity with Multiple Objects

Prompt: `"Pick the block closest to the blue bowl and place it inside the green bowl."`

Scenario: The environment contains three blocks (red, yellow, and blue) randomly placed on the workspace, along with a blue bowl and a green bowl. The user's intent is not tied to the color of the block, but rather to its relative distance from a reference object (the blue bowl).

Result: The system computes the (x, y) position of all objects, then evaluates the distance between each block and the blue bowl. It selects the block with the smallest distance and generates a pick-and-place instruction to move that block into the green bowl. This behavior is

only possible thanks to the positional input added to the prompt.

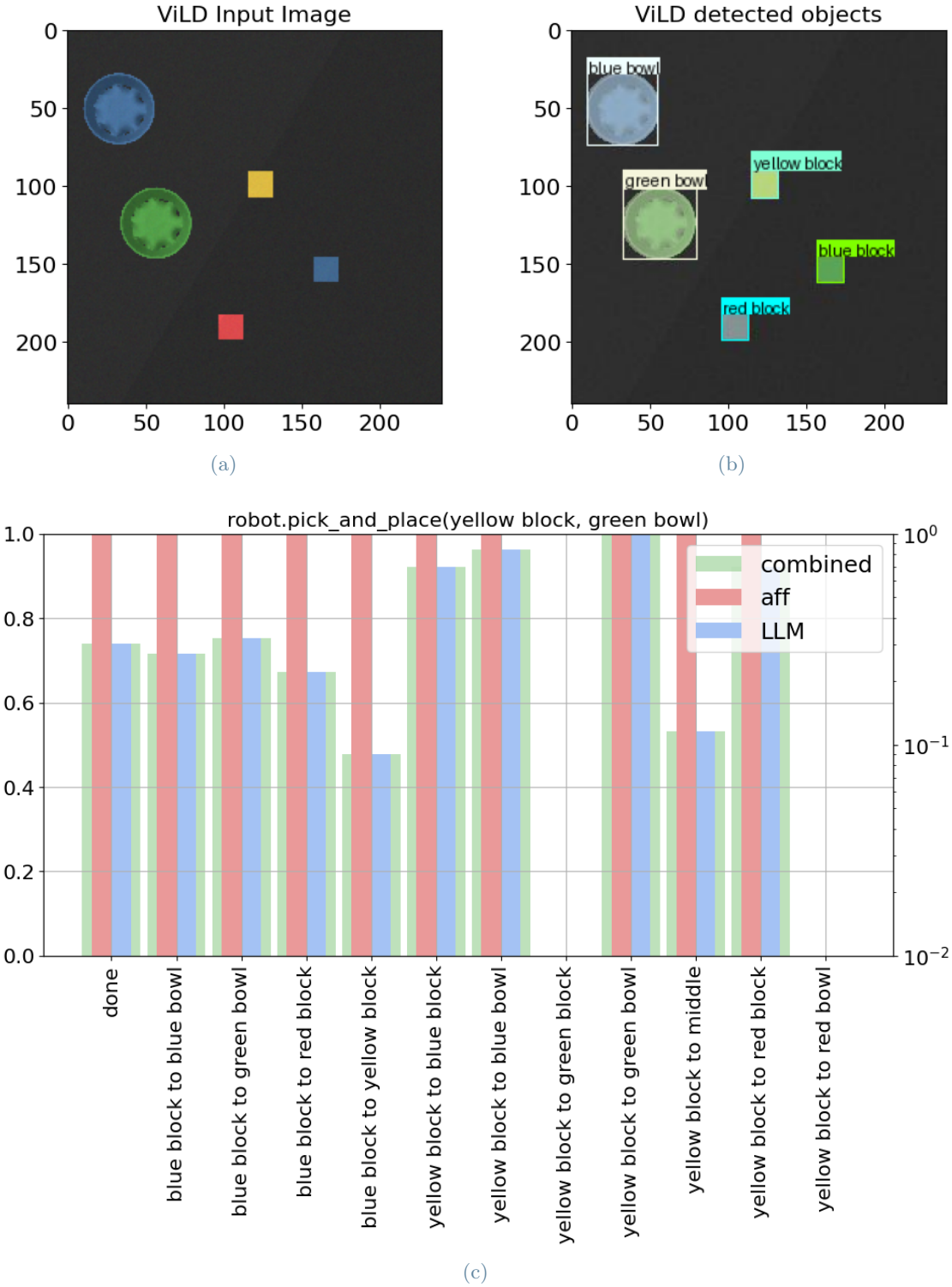


Figure 9: a) ViLD input image. b) ViLD output image. c) Affordance LLM and combined scores of the top 15 actions.

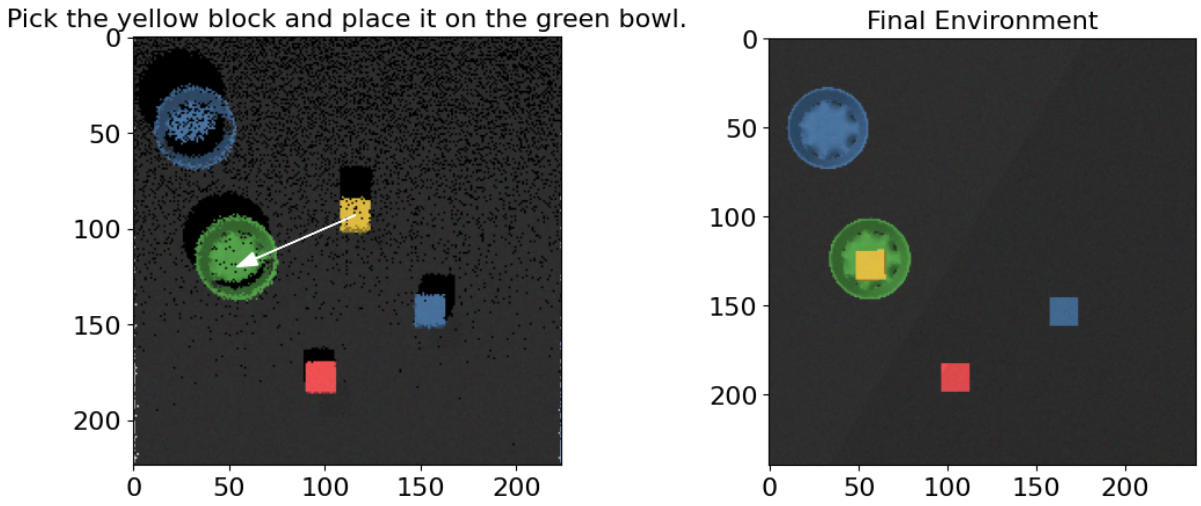


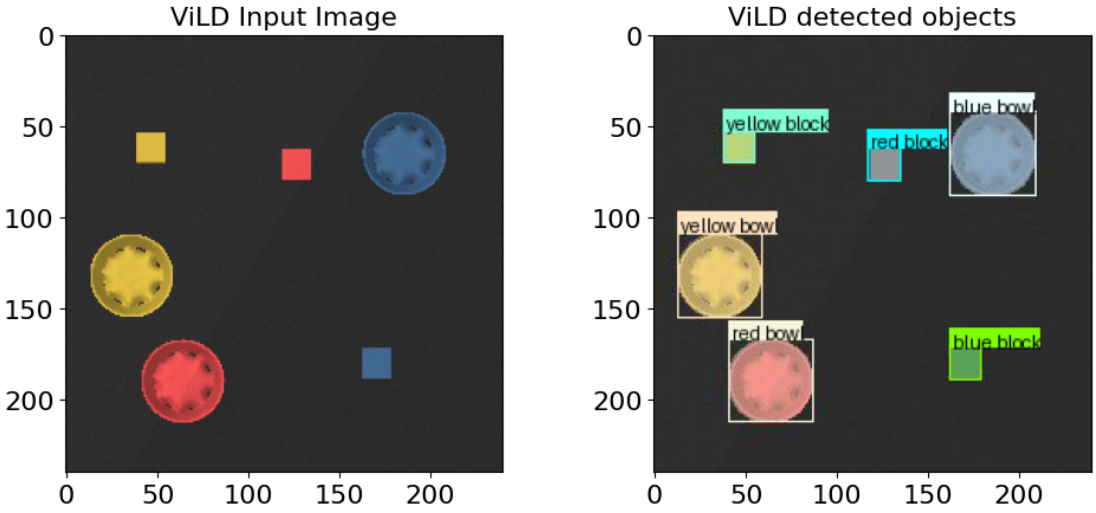
Figure 10: a) Selected primitive b) Final environment

Example 2: Right-to-Left Sorting with Color-Matched Placement

Prompt: "`Pick the blocks starting from the rightmost one and place each into the bowl with the same color.`"

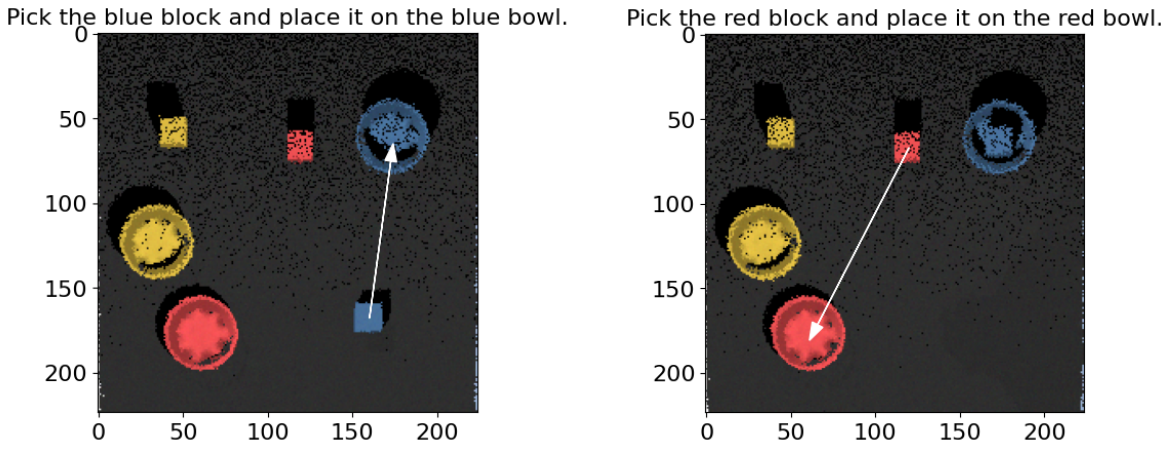
Scenario: The environment contains three colored blocks (yellow, blue, and red) and three bowls with matching colors (yellow, blue, and red), randomly arranged on the workspace. The user instructs the robot to act in a specific spatial order (right to left) and to place each block into the corresponding bowl based on color.

Result: The system first computes the (x, y) coordinates of all blocks and sorts them in descending order of their x position. It then iterates over the sorted list and, for each block, identifies the bowl with the same color label. For every block-bowl pair, it generates a pick-and-place action accordingly. This task combines spatial ordering with semantic matching, and it would be impossible to resolve correctly without positional data and object class information integrated into the prompt.



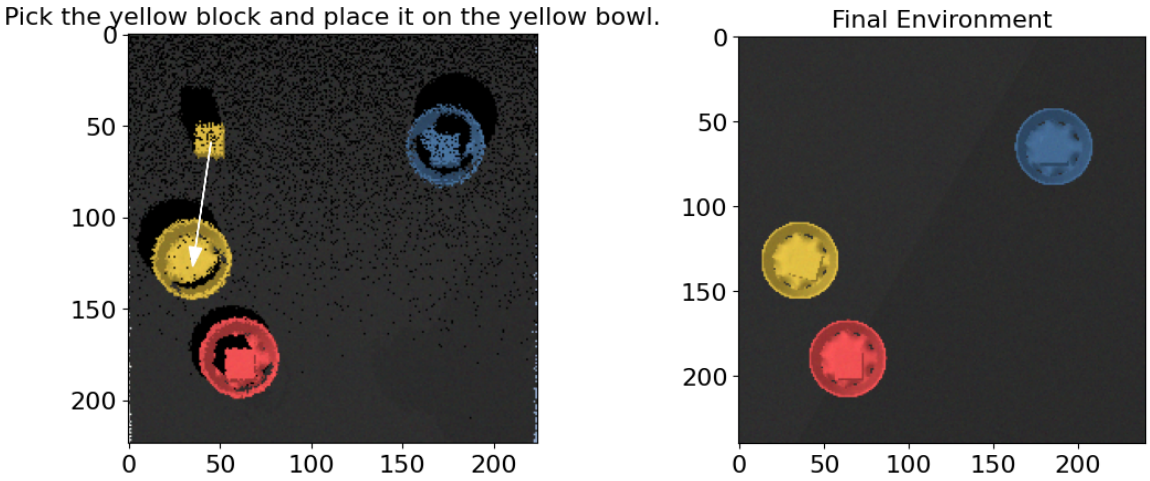
(a) (b)

Figure 11: a) ViLD input image. b) ViLD output image.



(a) (b)

Figure 12: a) First selected primitive. b) Second selected primitive.



(a) (b)

Figure 13: a) Third selected primitive b) Final environment

3.4. Advantages and Limitations

The integration of object positional information into the prompt enhances the model's ability to understand and resolve spatial references in a way that aligns more closely with human instructions. This modification enables the LLM to interpret and execute a broader class of tasks that rely not only on the presence of specific objects but also on their geometric arrangement within the scene.

By grounding language understanding in spatial data, the system is capable of reasoning over distances, relative positions, and conditional arrangements, all of which are crucial for performing realistic manipulation tasks. Examples such as "pick the object closest to the blue bowl" or "stack the blocks from right to left" become tractable only when the model has access to accurate (x, y) coordinates for each object in the scene.

However, this approach depends on several assumptions: namely, that ViLD reliably detects and segments the objects, and that the camera is properly calibrated for coordinate transformation. Inaccurate segmentation or occlusion can lead to errors in spatial reasoning. Nonetheless, our experimental observations suggest that the inclusion of positional inputs consistently leads to more grounded, contextually appropriate decisions, especially in environments with multiple similar objects.

In summary, by incorporating spatial information extracted via ViLD into the SayCan framework, we bridge the gap between semantic intent and spatial execution. This enhancement makes the system significantly more flexible, robust, and capable of performing complex, spatially-informed tasks in a simulated environment.

4 | Feedback and Plan Adjustment

1. Motivation

In the original robotic frameworks the environment is described once at initialization and remains static. However, when dealing with dynamic environments, like pick-and-place manipulation, this assumption fails. Objects change location after each step, and reasoning on a stale scene description compromises performance.

This limitation made the reasoning process of the LLM less grounded, as the scene could change significantly after each action, leading to: repeated or invalid actions, choices based on outdated spatial configurations, failure in multi-step planning under dynamic conditions.

2. Proposed Framework

In our modified implementation, a feedback loop is introduced:

- ViLD detects the objects present in the scene from the captured image.
- The detected bounding boxes are converted into 3D coordinates using the xyz map.
- The Scene description generator produces an updated textual description of the environment based on object categories and positions.
- The movement options are regenerated (e.g., pick and place pairs) according to the current state of the scene.
- With the updated LLM and Affordance Score each possible action is evaluated, comparing semantic reasoning (LLM) and physical feasibility (affordance).
- **C**LIPort executes the highest-scoring action and a new feedback iteration starts.

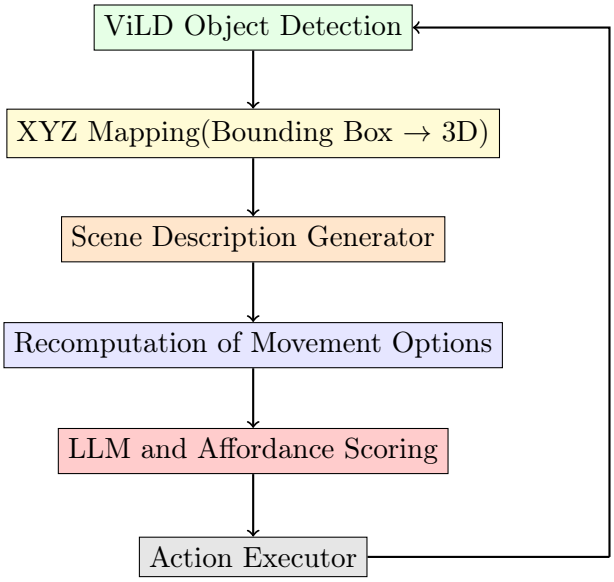


Figure 14: Closed-loop perception-language-action feedback system.

4.1. Implementation

Listing 4.1: Feedback Loop code

```
while not selected_task == termination_string:
    num_tasks += 1
    if num_tasks > max_tasks:
        break
    options_begin = make_options(PICK_TARGETS, PLACE_TARGETS,
       termination_string=termination_string)
    imageio.imsave(image_path, env.get_camera_image_top())
    found_objects, obj_pos_string = vild(image_path,
       category_name_string, vild_params, plot_on=True)

    print('\nFound Objects and positions from Vild: ', obj_pos_string)

    affordance_scores = affordance_scoring(options_begin, found_objects,
        block_name="box", bowl_name="circle", verbose=False)
    scene_description = build_scene_description(found_objects)
    env_description = 'BEGIN SCENE DESCRIPTION:\n' + scene_description +
        '\n' + \'The object are placed on a table of dimensions 0.6 and
       0.6. ' + \'Top-left corner is at (-0.25, -0.25). Top-right corner
        is at (0.25, -0.25). ' + \'Bottom-left corner is at (-0.25,
       -0.75). Bottom-right corner is at (0.25, -0.75)' + \
       obj_pos_string + '\nEND SCENE DESCRIPTION.\n'
    commands_string = commands_string + env_description
    gpt3_prompt = gpt3_context + call_law_start\n" + env_description
    LLM_CACHE = {}
    llm_scores, res = gpt3_scoring(query, gpt3_prompt, commands_string,
       options_begin, model="gpt-4o-mini", temperature=1.2, top_logprobs
       =5, verbose=True)
    normalized_llm_scores = normalize_scores(llm_scores)
    combined_scores = {}
    for option, llm_score in normalized_llm_scores.items():
        if option in affordance_scores:
            affordance_score = affordance_scores[option]
            combined_scores[option] = llm_score * affordance_score

    if combined_scores:
        selected_task = max(combined_scores, key=combined_scores.get)
```

The code defines an iterative feedback-driven control loop where the robotic agent interacts with its environment under the guidance of a Large Language Model (LLM). At the core of this implementation is a perception-reasoning-action-feedback pipeline, which ensures that every decision the agent makes is grounded in the current state of the environment.

At each iteration, the environment is first observed using an overhead camera. The captured RGB image is processed by ViLD, a vision-language object detector, to identify the visible objects and their positions. Based on this scene perception, the set of possible actions (pick and place combinations) is regenerated from scratch. Affordance scores, which indicate how physically feasible or meaningful each action is, are recalculated using this updated information.

However, the most substantial innovation in this system lies in the feedback mechanism applied to the LLM's decision-making process. Unlike traditional approaches where the language model is queried only once at the beginning of execution, here the LLM is prompted anew at every step

with an updated view of the world. This includes the list of detected objects, their positions, and an optional textual scene description. The prompt is rebuilt after each action to reflect what has changed in the environment, objects that have been moved, removed, or added, as well as spatial shifts. This modification transforms the LLM from a static planner into a dynamic agent capable of adaptive reasoning.

Moreover, to enhance the diversity and contextual sensitivity of the language model's responses, the GPT model is queried with a modified temperature setting (in this case, set to 1.2). A higher temperature allows the model to explore a wider range of possibilities, which is particularly useful when dealing with ambiguous or evolving environments. This configuration helps the LLM to avoid deterministic or overly repetitive responses, promoting more flexible behavior.

Once the updated prompt is constructed, the model scores all available options. These language-based scores are then normalized and combined with the affordance scores to produce a final ranking of actions. The action with the highest combined score is selected and executed in the environment. Crucially, this entire reasoning process is repeated at every iteration. The agent doesn't rely on a fixed plan; instead, it continuously observes the effects of its own actions and re-evaluates the scene before deciding what to do next. This closes the loop between perception and reasoning, making the system more robust, adaptive, and context-aware.

4.2. Advantages of Iterative Feedback

Grounded Reasoning. The LLM can reason based on the real-time configuration of the environment, ensuring that its next step is context-aware. This reduces hallucinations and logically inconsistent plans.

Dynamic Recovery. If an action fails or an object is displaced unexpectedly (e.g., during manipulation), the updated input allows the system to recover gracefully by adapting the next planned action.

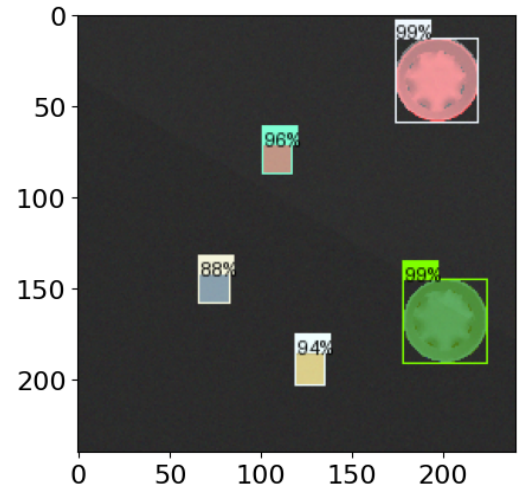
Improved Multi-Step Planning. Each new iteration allows the system to re-assess goals. For example, if "stack the red block on the blue block" was interrupted, the next action will consider whether:

- the blocks are still reachable,
- the first part of the goal has already been fulfilled,
- a different order is now more feasible.

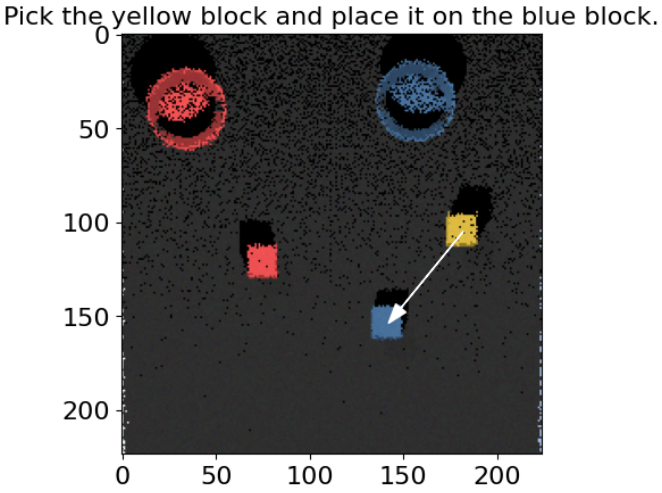
Better Alignment with Human Reasoning. The feedback enables the robot to exhibit more flexible, human-like decision-making.

4.3. Example 1

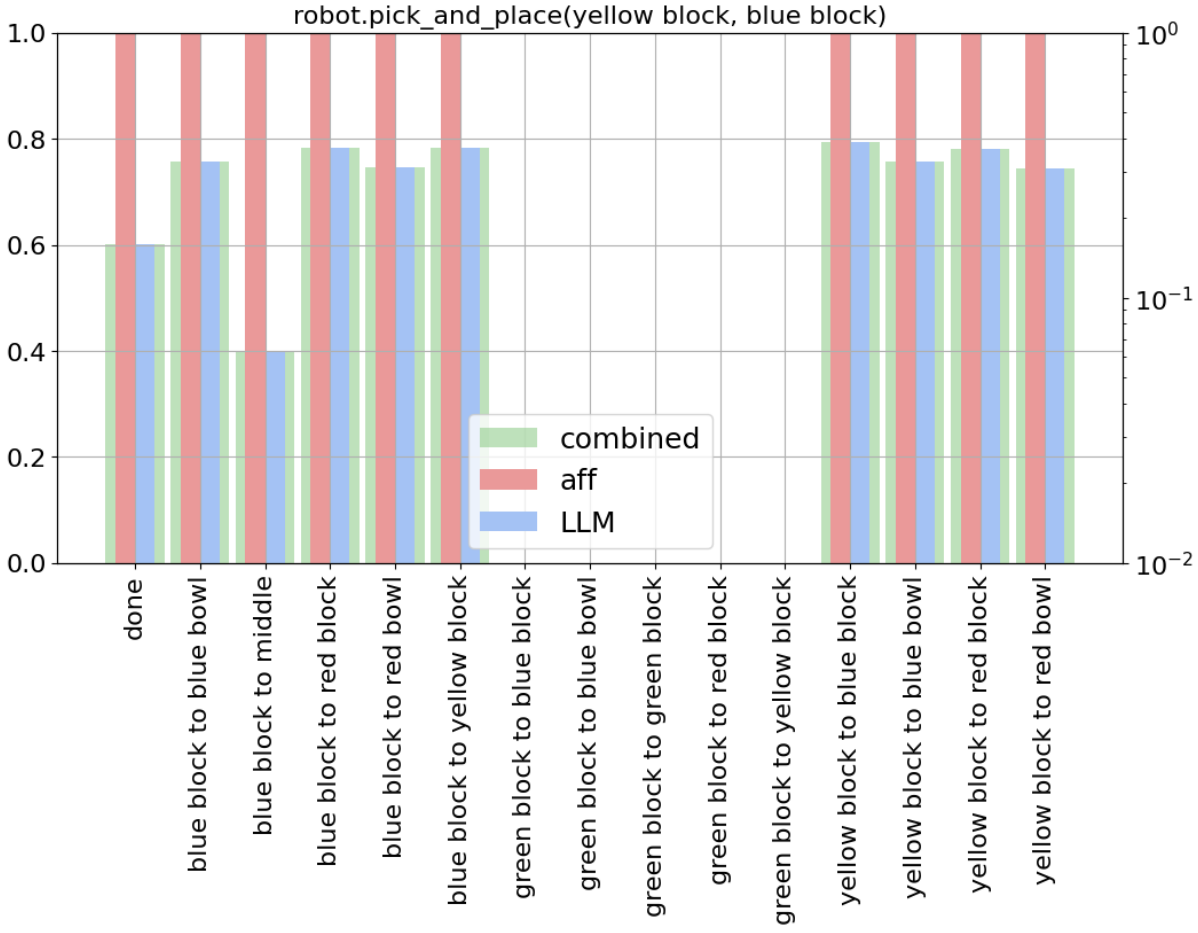
QUERY: stack all the blocks



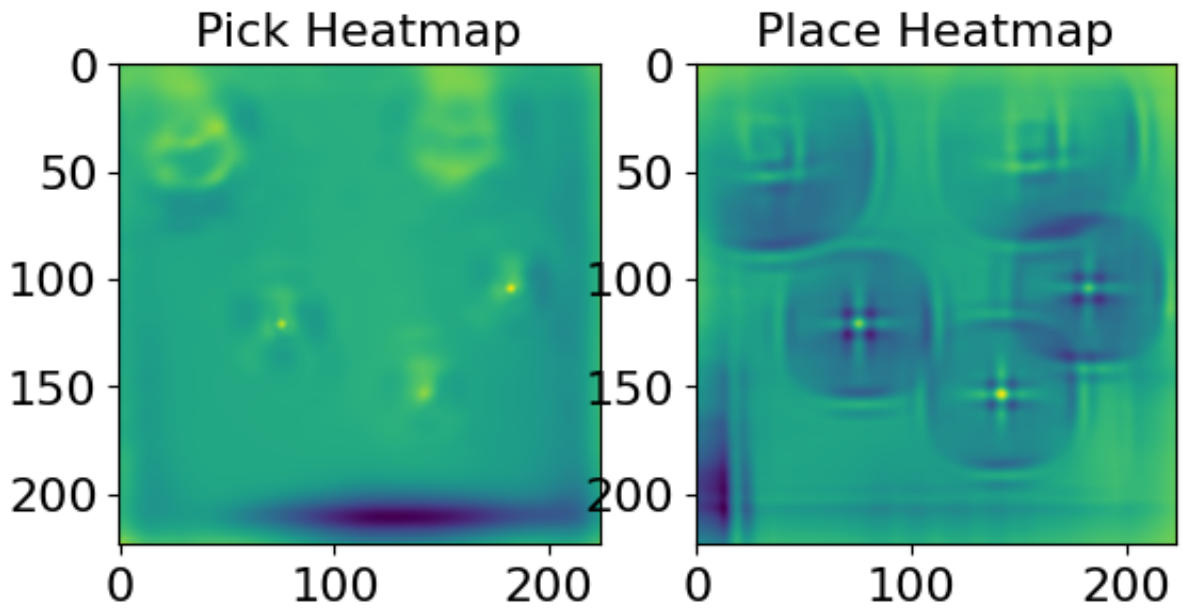
(a)



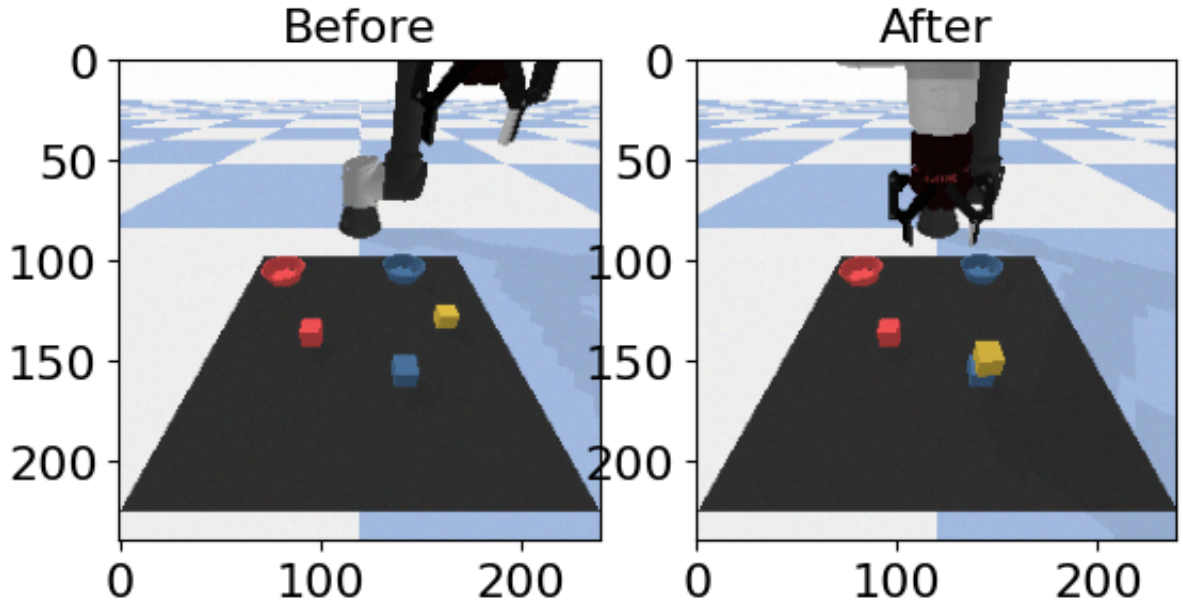
(b)



(c)

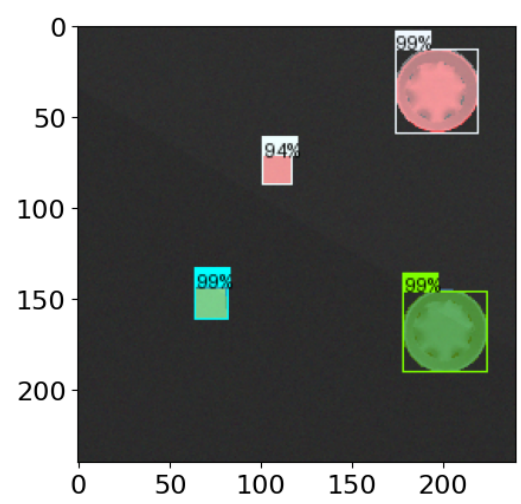


(d)

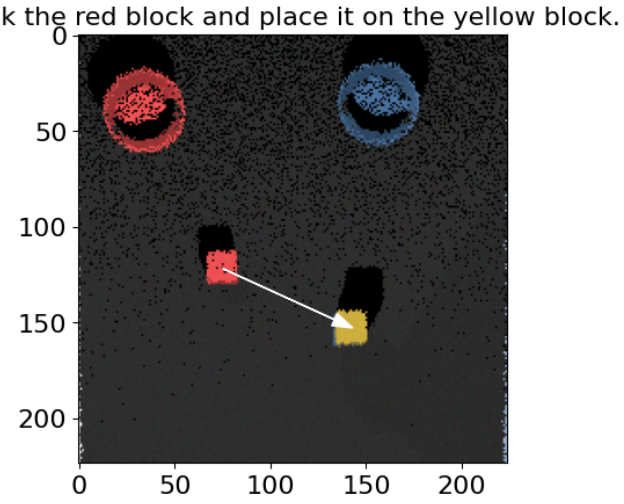


(e)

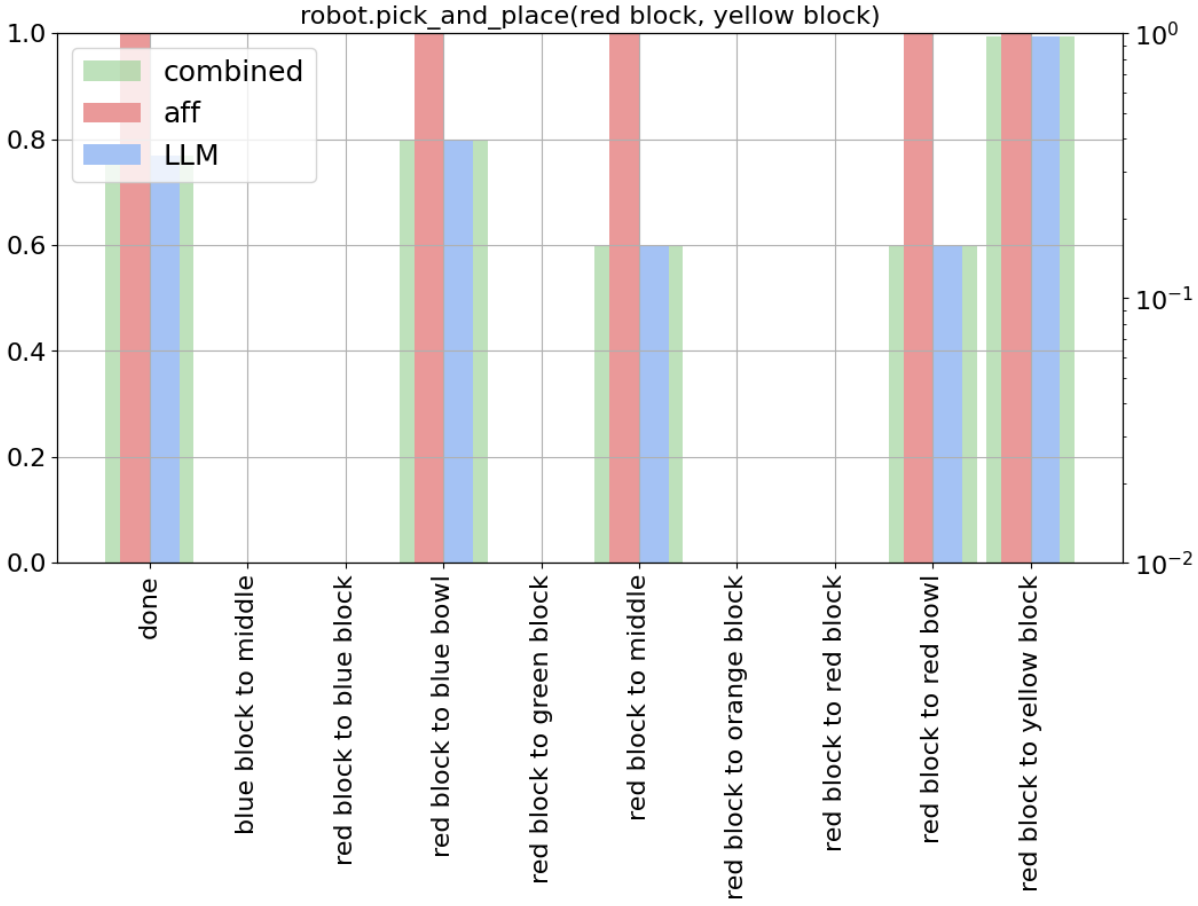
Figure 15: First movement of stacking: a) ViLD object recognition. b) Action performed. c) Affordance LLM and combined scores of the top 15 actions. d) CLIPort pick and place heatmaps. e) Environment before and after the action performed


0
50
100
150
200
99%
94%
99%
99%

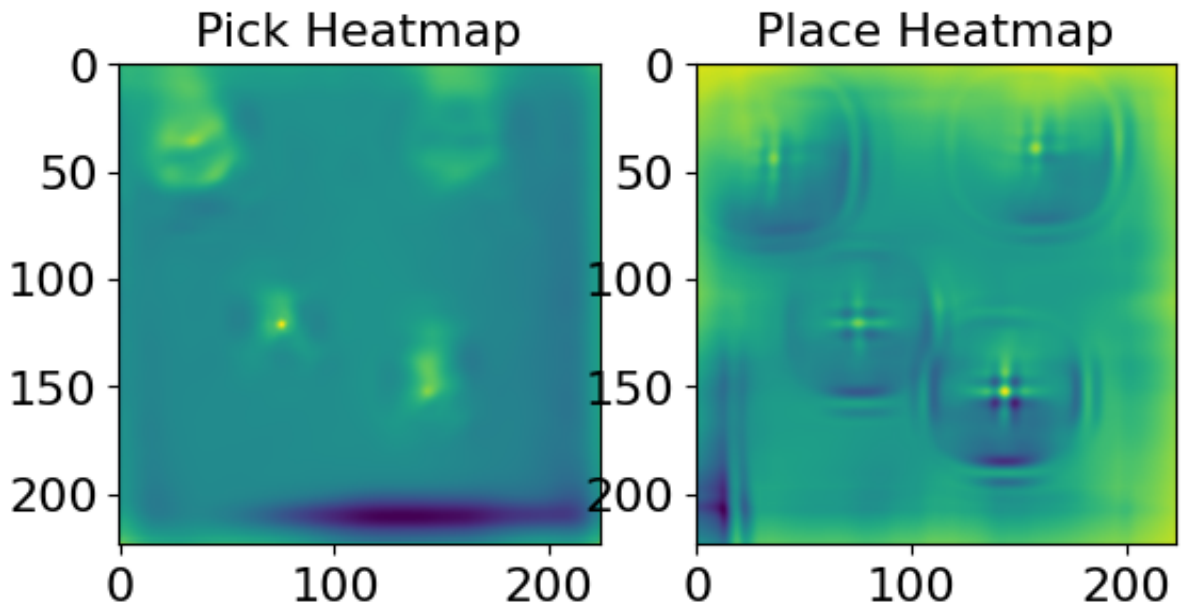
(a)


Pick the red block and place it on the yellow block.
0
50
100
150
200

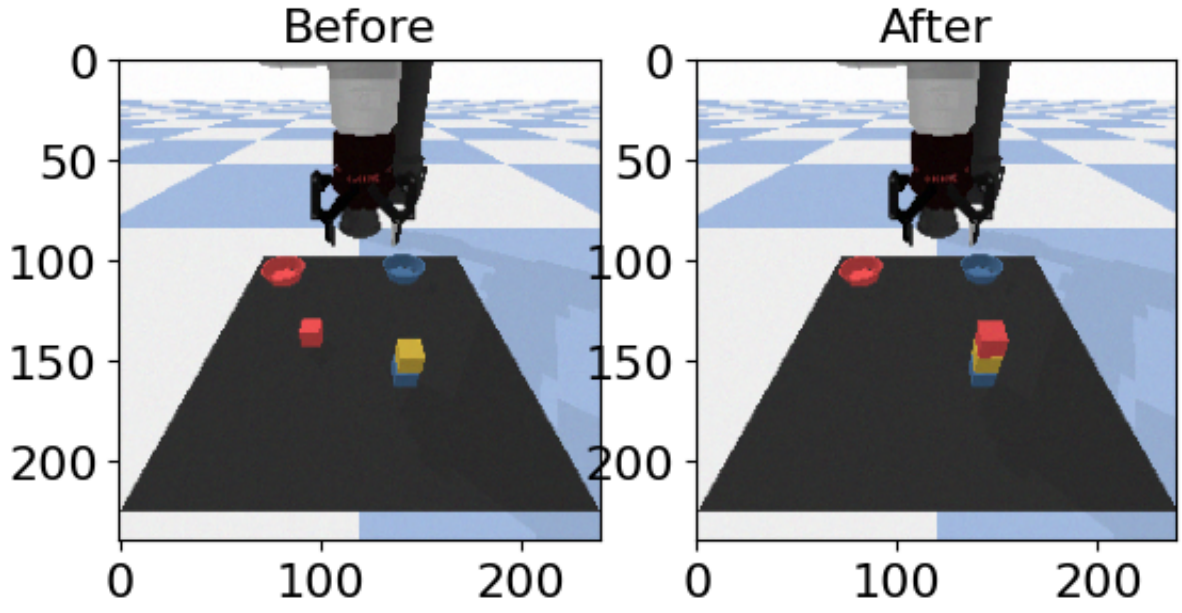
(b)


robot.pick_and_place(red block, yellow block)
combined
aff
LLM
1.0
0.8
0.6
0.4
0.2
0.0
10^0
10^{-1}
10^{-2}
done
blue block to middle
red block to blue block
red block to blue bowl
red block to green block
red block to middle
red block to orange block
red block to red block
red block to red bowl
red block to yellow block

(c)



(d)



(e)

Figure 16: Second movement of stacking: a) ViLD object recognition. b) Action performed. c) Affordance LLM and combined scores of the top 15 actions. d) CLIPort pick and place heatmaps. e) Environment before and after the action performed

4.4. Example 2

QUERY: clockwise, move the block through all the corners (starting from top left). Then put it in the bowl.

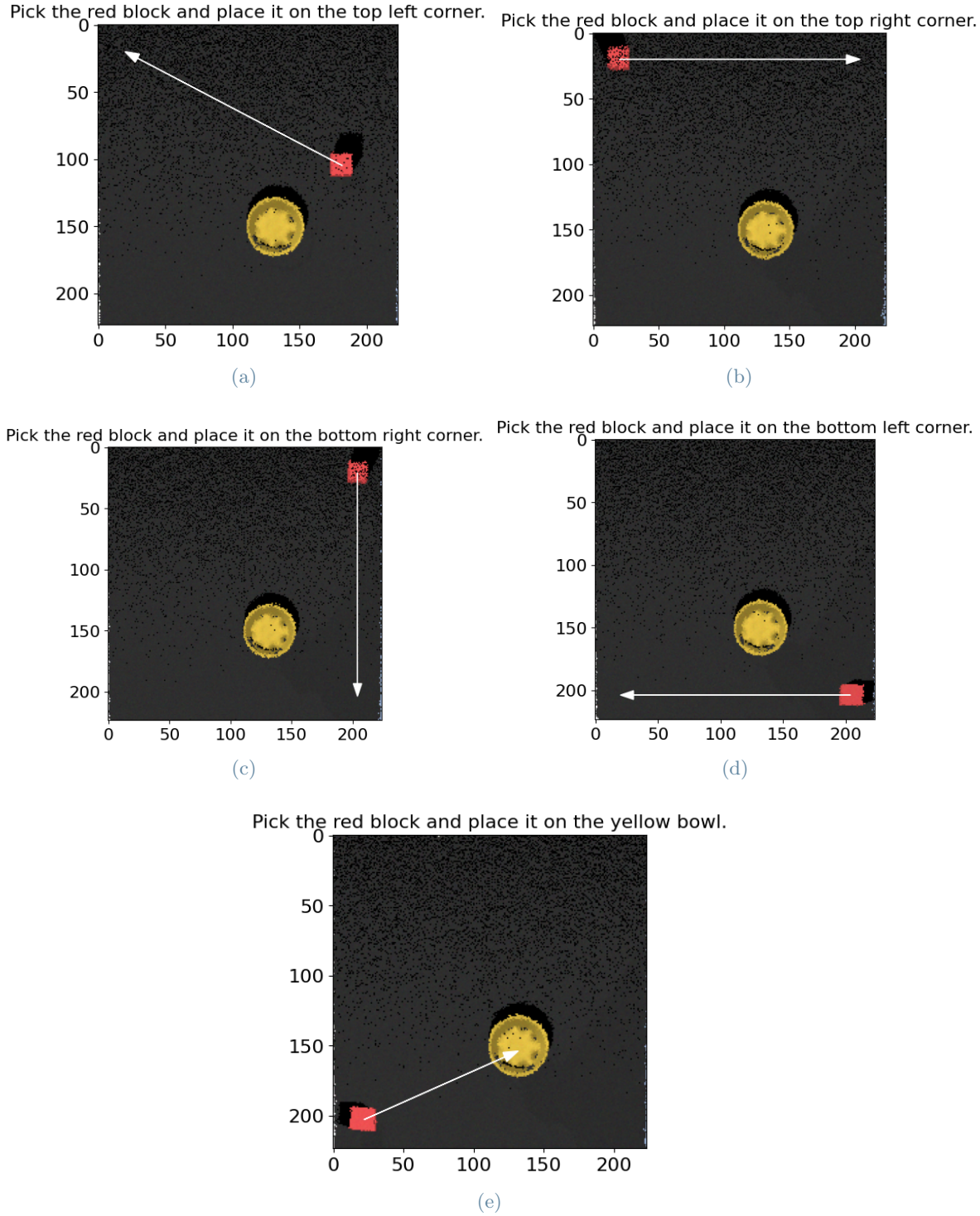


Figure 17: Actions performed a) Move the block to top left corner.b) Move the block to top right corner. c) Move the block to bottom right corner. d) Move the block to bottom left corner. e) Move the block to yellow bowl.

5 | Conclusions

This project successfully extended the SayCan framework by integrating spatial grounding, environment adaptability, and object recognition enhancements into the decision-making pipeline of a robotic agent. By doing so, we significantly improved the alignment between high-level natural language commands and low-level robotic actions in a dynamic simulation environment.

A first core contribution was the introduction of new object types and geometric shapes into the simulated workspace. The expanded object set, allows for a richer and more diverse testing environment. These additions, combined with reliable ViLD-based object detection and classification, enabled the system to interpret and recognize the new objects.

The second major enhancement was the spatial grounding of LLM prompts. By incorporating real-time (x, y) coordinates of each object into the language input, the model could reason over spatial relations—such as proximity, order, and orientation—which are essential for tasks involving more abstract or implicit spatial instructions. The experiments showed that tasks previously ambiguous or intractable, such as "pick the object closest to the bowl" or "place the blocks from right to left," became solvable through this enriched prompting method.

Finally, we introduced a closed-loop feedback mechanism that re-evaluates the scene and regenerates the plan after each action. This iterative process ensures that the system remains synchronized with the environment's state, avoiding logical inconsistencies and improving robustness. The continuous update of both the visual scene and the LLM's context allows the robot to adapt to failed actions, unexpected disturbances, or shifting goals in a human-like fashion.

Overall, the proposed extensions make SayCan a more versatile, intelligent, and reactive framework for vision-language-action integration in robotics. While some limitations remain, such as dependency on accurate object detection and fixed camera calibration, this work demonstrates the feasibility and value of integrating perception, reasoning, and action through LLM-guided planning. These contributions pave the way for future work on real-world deployment, active perception, and deeper integration of multi modal feedback for robotic agents capable of more autonomous and context-aware behavior.